DAX Full Notes

Complete study notes on DAX (Data Analysis Expressions) for Power BI, Excel Power Pivot and SSAS Tabular.

1. What is DAX?

DAX is a formula and query language used in Power BI, Power Pivot (Excel) and SQL Server Analysis Services (Tabular). It is built for working with relational data models: tables, relationships, columns and measures.

Key points:

- DAX is a **functional language**: every formula is an expression that returns a value.
- It is **not** a programming language with loops or statements; it evaluates expressions in a context.
- It is similar in syntax to Excel formulas but operates on **columns and tables**, not cells.
- Two main uses: **calculated columns / tables / measures** (modeling) and **DAX queries** (EVALUATE).

2. Syntax Basics

```
Total Sales = SUM(Sales[SalesAmount])
```

- `Total Sales` = name of the measure
- `=` = operator that starts the formula
- `SUM` = function
- `Sales[SalesAmount]` = table name + column name

Rules:

- Table names with spaces need single quotes: `'Product Category'[Name]`.
- Column references are written `Table[Column]`; measure references are written `[Measure]` without a table.
- Comments: `//` single line, `--` single line, `/* ... */` multi-line.
- DAX is not case sensitive for function names.
- Use `VAR ... RETURN` for readable, efficient formulas.

Operators

Type	Operators
Arithmetic	`+ - * / ^`
Comparison	`= == <> > >= < <=`
Text concatenation	`&`
Logical	`&&` (AND), `

`==` is a strict comparison (type-sensitive and treats BLANK differently from 0 or empty string); `=` is the normal comparison.

Data Types

Whole Number (Int64), Decimal Number (Double), Fixed Decimal / Currency, Boolean (TRUE/FALSE), Text, Date/Time, Binary, and BLANK (which is a value, not an error).

3. Where DAX is Used

1. **Calculated columns**: computed row by row at data refresh and stored in the model (uses memory).
2. **Measures**: computed at query time, based on the filter context (no storage; dynamic).
3. **Calculated tables**: whole tables created with a DAX expression, e.g. a Date table.
4. **Row-level security (RLS)**: filter expressions that restrict data by role.
5. **Queries**: `EVALUATE` statements in DAX Studio, SSMS or the Power BI query view.

Calculated Column vs Measure

Aspect	Calculated Column	Measure
Evaluated	At refresh, per row	At query time, per cell
Context	Row context	Filter context
Storage	Stored in model	Not stored
Used in	Rows, columns, slicers, filters	Values area
Best for	Categorization, slicers, relationships	Aggregations, ratios, KPIs

Rule of thumb: if you aggregate, use a measure. If you need to slice or filter by it, use a column.

4. Evaluation Context (The Most Important Concept)

Every DAX expression is evaluated inside a context. Understanding context is the key to understanding DAX.

4.1 Filter Context

The set of filters applied to the data model when a calculation runs. It comes from:

- Rows and columns in a visual or pivot table
- Slicers and page/report/visual-level filters
- Filters introduced by CALCULATE
- Relationship propagation (filters flow from the one side to the many side)

Example: a matrix showing Sales by Year. For the 2024 row, the filter context is `Date[Year] = 2024`, so `SUM(Sales[Amount])` returns only 2024 sales.

4.2 Row Context

Iterates a table one row at a time. It exists:

- In calculated columns (automatically)
- Inside iterator functions (SUMX, AVERAGEX, FILTER, ADDCOLUMNS, etc.)

Row context lets you refer to the current row's column values, but it **does not filter** other tables automatically.

4.3 Context Transition

When CALCULATE (or a measure reference) is evaluated inside a row context, the row context is converted into an equivalent filter context. This is why calling a measure inside an iterator works row by row.

```
Customer Sales Column = CALCULATE(SUM(Sales[Amount]))
```

In a calculated column on Customer, this returns that customer's sales because the row context became a filter on the customer.

4.4 Query Context vs Filter Context

Query context is the initial filter context created by the visual (rows, columns, slicers). CALCULATE modifies it to produce the final filter context used by the expression.

5. Aggregation Functions

Function	Purpose
SUM	Sum of a column
AVERAGE	Average of a column
MIN / MAX	Minimum / maximum
COUNT	Counts numeric (non-blank) values
COUNTA	Counts non-blank values of any type
COUNTROWS	Counts rows in a table
DISTINCTCOUNT	Counts distinct values
COUNTBLANK	Counts blank values

Iterator (X) Functions

SUMX, AVERAGEX, MINX, MAXX, COUNTX, RANKX, PRODUCTX, CONCATENATEX.

They take a table and an expression, evaluate the expression for each row, then aggregate.

```
Total Revenue = SUMX(Sales, Sales[Quantity] * Sales[UnitPrice])
```

Use SUMX when the value requires a row-level calculation before summing. `SUM(Sales[Quantity]) * SUM(Sales[UnitPrice])` is wrong because it multiplies totals.

6. CALCULATE and CALCULATETABLE

The most powerful DAX function. It evaluates an expression in a modified filter context.

```
CALCULATE( <expression>, <filter1>, <filter2>, ... )
```

```
Red Sales = CALCULATE(SUM(Sales[Amount]), Product[Color] = "Red")
```

How it works:

1. Copies the current filter context.
2. Applies the filter arguments (overriding existing filters on the same columns).
3. Evaluates the expression in the new context.

Important rules:

- A boolean filter like `Product[Color] = "Red"` is shorthand for `FILTER(ALL(Product[Color]), Product[Color] = "Red")`.
- Filters on the **same column** replace existing filters; filters on different columns combine (AND).
- Multiple filter arguments are combined with AND. For OR, use `||` on the same column, `IN`, or separate logic.
- CALCULATETABLE is the same idea but returns a table.

Filter Modifiers

Function	Effect
ALL	Removes filters from a table or columns
ALLEXCEPT	Removes all filters except on the given columns
ALLSELECTED	Removes filters inside the visual but keeps outside (slicer) filters
REMOVEFILTERS	Same as ALL when used as a CALCULATE modifier (clearer name)
KEEPFILTERS	Adds a filter without overwriting the existing one
VALUES	Returns distinct values currently visible (with blank row if any)
USERELATIONSHIP	Activates an inactive relationship for the calculation
CROSSFILTER	Changes the cross-filter direction for the calculation
TREATAS	Applies a table of values as filters on other columns

Percent of Total Examples

```
% of Grand Total =
DIVIDE( [Total Sales], CALCULATE([Total Sales], ALL(Sales)) )

% of Category =
DIVIDE( [Total Sales], CALCULATE([Total Sales], ALLEXCEPT(Product,
Product[Category])) )

% of Visible Total =
DIVIDE( [Total Sales], CALCULATE([Total Sales], ALLSELECTED()) )
```

7. Filter Functions

Function	Description
FILTER(table, condition)	Returns a table of rows where the condition is true (iterator)
ALL(table or columns)	Returns all rows / values ignoring filters
VALUES(column)	Distinct values in the current filter context
DISTINCT(column)	Distinct values (no extra blank row from broken relationships)
SELECTEDVALUE(col, alt)	Returns the value if only one is selected, else the alternate
HASONEVALUE(col)	TRUE if exactly one value is in context
ISFILTERED / ISCROSSFILTERED	Tests whether a column is being filtered
TOPN(n, table, expr, order)	Returns the top n rows
RELATED(column)	Fetches a value from the one side of a relationship (row context)
RELATEDTABLE(table)	Returns related rows from the many side
LOOKUPVALUE	Looks up a value without needing a relationship

Performance tip: prefer simple column filters inside CALCULATE (`Product[Color] = "Red"`) over `FILTER(Product, ...)` because the engine optimizes them better.

8. Logical and Conditional Functions

- **IF(condition, true, [false])**
- **IFERROR(value, value_if_error)**
- **SWITCH(expression, value1, result1, ..., [else])**
- **AND, OR, NOT**, and the operators `&&`, `||`
- **TRUE(), FALSE()**
- **COALESCE(a, b, ...)**: first non-blank value
- **ISBLANK, ISERROR, ISNUMBER, ISTEXT**

```
Sales Band =
SWITCH( TRUE(),
    [Total Sales] >= 100000, "High",
    [Total Sales] >= 50000,  "Medium",
    "Low" )
```

`SWITCH(TRUE(), ...)` is the standard pattern for multiple conditions.

9. Text Functions

Function	Example / Use
CONCATENATE, `&`	Join text
CONCATENATEX	Join values from a table with a delimiter
LEFT, RIGHT, MID	Extract characters
LEN	Length of text
UPPER, LOWER	Change case
TRIM	Remove extra spaces
SUBSTITUTE, REPLACE	Replace text
SEARCH, FIND	Locate text (SEARCH is case-insensitive, FIND is case-sensitive)
FORMAT(value, format)	Convert to text with a format string
VALUE	Text to number
REPT	Repeat text

```
Full Name = Customer[FirstName] & " " & Customer[LastName]
Month Label = FORMAT(Date[Date], "MMM YYYY")
```

10. Math and Statistical Functions

ABS, ROUND, ROUNDUP, ROUNDDOWN, INT, TRUNC, MOD, POWER, SQRT, DIVIDE, RAND, RANDBETWEEN, CEILING, FLOOR.

Statistical: MEDIAN, PERCENTILE.INC, PERCENTILE.EXC, STDEV.S, STDEV.P, VAR.S, VAR.P, RANKX, GEOMEAN.

DIVIDE(numerator, denominator, [alternate]) is preferred over `/` because it safely handles division by zero.

RANKX

```
Product Rank = RANKX( ALL(Product[Name]), [Total Sales], , DESC, DENSE )
```

Arguments: table, expression, value, order (ASC/DESC), ties (SKIP/DENSE).

11. Date and Time Functions

Basic

DATE, TIME, TODAY, NOW, YEAR, MONTH, DAY, HOUR, MINUTE, SECOND, WEEKDAY, WEEKNUM, EOMONTH, EDATE, DATEDIFF, DATEVALUE, FORMAT.

```
DATEDIFF(Customer[DOB], TODAY(), YEAR)
```

The Date Table

Time intelligence requires a proper date table:

- Contains **contiguous dates** (no gaps), covering all dates in your fact tables, full years.
- Has one row per date; the Date column is unique and of type Date.
- Marked as a Date table (Table tools, Mark as date table).
- Related to the fact table's date column.

```
Date =
ADDCOLUMNS(
    CALENDAR(DATE(2020,1,1), DATE(2030,12,31)),
    "Year", YEAR([Date]),
    "Month No", MONTH([Date]),
    "Month", FORMAT([Date], "MMMM"),
    "Quarter", "Q" & QUARTER([Date]),
    "Weekday", FORMAT([Date], "dddd")
)
```

CALENDAR(start, end) creates a date range; CALENDARAUTO() builds it automatically from the model's dates.

12. Time Intelligence Functions

These work on a marked Date table and return modified date filters, usually inside CALCULATE.

Category	Functions
To-date totals	TOTALYTD, TOTALQTD, TOTALMTD, DATESYTD, DATESQTD, DATESMTD
Prior periods	SAMEPERIODLASTYEAR, PREVIOUSDAY / MONTH / QUARTER / YEAR, DATEADD, PARALLELPERIOD
Next periods	NEXTDAY / MONTH / QUARTER / YEAR
Period boundaries	STARTOFMONTH / QUARTER / YEAR, ENDOFMONTH / QUARTER / YEAR, FIRSTDATE, LASTDATE
Ranges	DATESBETWEEN, DATESINPERIOD
Opening/closing balance	OPENINGBALANCEMONTH, CLOSINGBALANCEMONTH, etc.

```
Sales YTD = TOTALYTD([Total Sales], 'Date'[Date])

Sales PY = CALCULATE([Total Sales], SAMEPERIODLASTYEAR('Date'[Date]))

YoY Growth % =
DIVIDE( [Total Sales] - [Sales PY], [Sales PY] )

Sales Last 3 Months =
CALCULATE([Total Sales], DATESINPERIOD('Date'[Date], MAX('Date'[Date]), -3,
MONTH))

Running Total =
CALCULATE([Total Sales], FILTER(ALL('Date'[Date]), 'Date'[Date] <=
MAX('Date'[Date])))
```

Use DATEADD for flexible shifting: `DATEADD('Date'[Date], -1, YEAR)`, `-1, MONTH`, `-1, QUARTER`.

13. Table Functions

Function	Purpose
SUMMARIZE	Group by columns (like SQL GROUP BY); avoid adding calculations inside it
SUMMARIZECOLUMNS	Efficient grouping with calculations; used by Power BI visuals and queries
ADDCOLUMNS	Adds calculated columns to a table
SELECTCOLUMNS	Returns selected / renamed columns
GROUPBY	Groups with iterator-style aggregations
UNION, INTERSECT, EXCEPT	Set operations
CROSSJOIN	Cartesian product
NATURALINNERJOIN, NATURALLEFTOUTERJOIN	Joins on matching columns
GENERATE, GENERATEALL	Row-by-row table generation (like CROSS APPLY)
GENERATESERIES	Creates a numeric sequence (useful for what-if parameters)

Function	Purpose
ROW	Single-row table
DATATABLE	Creates a table with hard-coded data
TOPN	Top N rows
DISTINCT, VALUES, ALL	Distinct / all values

```
Top 5 Products =
TOPN(5, ALL(Product[Name]), [Total Sales], DESC)

Sales by Category =
SUMMARIZECOLUMNS( Product[Category], "Sales", [Total Sales] )
```

14. Information Functions

ISBLANK, ISERROR, ISLOGICAL, ISNUMBER, ISTEXT, ISEVEN, ISODD, CONTAINS, CONTAINSROW, USERNAME, USERPRINCIPALNAME, HASONEVALUE, HASONEFILTER, ISFILTERED, ISCROSSFILTERED, ISINSCOPE (used for hierarchies and subtotals).

15. Variables (VAR / RETURN)

```
YoY Growth % =
VAR CurrentSales = [Total Sales]
VAR PriorSales = CALCULATE([Total Sales], SAMEPERIODLASTYEAR('Date'[Date]))
RETURN
    DIVIDE( CurrentSales - PriorSales, PriorSales )
```

Benefits:

- **Readability**: break complex logic into named steps.
- **Performance**: a variable is evaluated once and reused.
- **Debugging**: return a variable to inspect its value.
- Variables are **constants within their scope**: they are evaluated in the context where they are defined (not where they are used), which matters when used inside CALCULATE.

16. Relationships and DAX

- Relationship types: one-to-many (most common), one-to-one, many-to-many.

- Filter direction: single (one side filters many side) or both.
- Only **one active** relationship between two tables; others are inactive and activated with USERELATIONSHIP.
- RELATED works from the many side to the one side; RELATEDTABLE goes the opposite way.
- Star schema (fact tables + dimension tables) is the recommended model design.

```
Ship Date Sales =
CALCULATE([Total Sales], USERELATIONSHIP(Sales[ShipDate], 'Date'[Date]))
```

17. Common Patterns

Distinct customers who purchased

```
Active Customers = DISTINCTCOUNT(Sales[CustomerID])
```

Average per group

```
Avg Sales per Customer = AVERAGEX(VALUES(Customer[CustomerID]), [Total
Sales])
```

Cumulative / running total

```
Cumulative Sales =
CALCULATE([Total Sales], FILTER(ALLSELECTED('Date'[Date]), 'Date'[Date] <=
MAX('Date'[Date])))
```

Dynamic title

```
Title = "Sales for " & SELECTEDVALUE(Product[Category], "All Categories")
```

Moving average (3 months)

```
3M Moving Avg =
AVERAGEX( DATESINPERIOD('Date'[Date], MAX('Date'[Date]), -3, MONTH), [Total
Sales] )
```

New vs returning customers

```
New Customers =
COUNTROWS(
    FILTER( VALUES(Customer[CustomerID]),
        CALCULATE(MIN(Sales[OrderDate])) >= MIN('Date'[Date]) ) )
```

Handling blanks

```
Sales = IF( ISBLANK([Total Sales]), 0, [Total Sales] )
-- or
Sales = [Total Sales] + 0
```

18. Performance Best Practices

1. Use a **star schema**; avoid wide, flat tables and unnecessary bidirectional filters.
2. Prefer **measures** over calculated columns for aggregations.
3. Use **variables** to avoid recalculating the same expression.
4. Use **DIVIDE** instead of `/`.
5. Filter **columns**, not whole tables, inside CALCULATE.
6. Avoid iterating large tables with nested iterators.
7. Reduce column cardinality (e.g. split datetime into date and time, round decimals).
8. Remove unused columns and disable auto date/time when you have your own Date table.
9. Use SELECTEDVALUE instead of `IF(HASONEVALUE(...), VALUES(...))`.
10. Use **DAX Studio** and **Performance Analyzer** to measure and optimize queries.

19. Common Mistakes

- Confusing row context and filter context.
- Using SUM on a product of columns instead of SUMX.
- Forgetting to mark the Date table or having gaps in dates, which breaks time intelligence.
- Putting measures where calculated columns are needed (and vice versa).
- Using FILTER on whole tables instead of columns.
- Ignoring blank rows created by unmatched relationships.
- Writing `Table[Column]` for measures (measures should be referenced as `[Measure]`).

- Circular dependencies between calculated columns or tables.

20. DAX Queries

```
EVALUATE
SUMMARIZECOLUMNS(
    'Date'[Year],
    Product[Category],
    "Sales", [Total Sales]
)
ORDER BY 'Date'[Year], [Sales] DESC
```

Structure: optional `DEFINE` (local measures/variables), `EVALUATE` (required, returns a table), optional `ORDER BY` and `START AT`.

Tools: DAX Studio, SSMS, Tabular Editor, Power BI Desktop's DAX query view.

21. Quick Reference Cheat Sheet

Goal	Use
Sum a column	SUM
Row-by-row calculation, then sum	SUMX
Change filter context	CALCULATE
Remove filters	ALL / REMOVEFILTERS / ALLEXCEPT
Respect slicers but ignore visual filters	ALLSELECTED
Safe division	DIVIDE
Single selected value	SELECTEDVALUE
Multiple conditions	SWITCH(TRUE(), ...)
Year-to-date	TOTALYTD / DATESYTD
Same period last year	SAMEPERIODLASTYEAR / DATEADD
Rank	RANKX
Top N	TOPN
Fetch value from related table	RELATED
Use inactive relationship	USERELATIONSHIP

Goal	Use
Intermediate values	VAR / RETURN

22. Interview Questions

1. What is the difference between a calculated column and a measure?
2. Explain row context and filter context.
3. What is context transition?
4. Difference between ALL, ALLEXCEPT and ALLSELECTED?
5. Difference between SUM and SUMX?
6. What does CALCULATE do and how does it handle existing filters?
7. Difference between VALUES and DISTINCT?
8. How do you implement YTD and previous-year comparisons?
9. Why do you need a Date table and what makes it valid?
10. When would you use USERELATIONSHIP?
11. Why are variables useful?
12. How do you optimize a slow DAX measure?

Notes by Aparana Varshney